

Task 1.1

--- Forking a team of 4 threads ---

[Master] Logical Rank: 0 of 4 | Native OS TID: 19976

[Worker] Logical Rank: 3 of 4 | Native OS TID: 20648

--- Joined thread team. Execution returned to serial master ---

--- Forking a team of 4 threads ---

[Master] Logical Rank: 0 of 4 | Native OS TID: 23948

[Worker] Logical Rank: 3 of 4 | Native OS TID: 26464

--- Joined thread team. Execution returned to serial master ---

--- Forking a team of 4 threads ---

[Master] Logical Rank: 0 of 4 | Native OS TID: 23580

[Worker] Logical Rank: 3 of 4 | Native OS TID: 2012

--- Joined thread team. Execution returned to serial master ---

--- Forking a team of 4 threads ---

[Master] Logical Rank: 0 of 4 | Native OS TID: 33364

[Worker] Logical Rank: 3 of 4 | Native OS TID: 25964

--- Joined thread team. Execution returned to serial master ---

--- Forking a team of 4 threads ---

[Master] Logical Rank: 0 of 4 | Native OS TID: 36164

[Worker] Logical Rank: 3 of 4 | Native OS TID: 11136

--- Joined thread team. Execution returned to serial master ---

--- Forking a team of 4 threads ---

[Master] Logical Rank: 0 of 4 | Native OS TID: 27544

[Worker] Logical Rank: 3 of 4 | Native OS TID: 18104

--- Joined thread team. Execution returned to serial master ---

--- Forking a team of 4 threads ---

[Master] Logical Rank: 0 of 4 | Native OS TID: 3976

[Worker] Logical Rank: 3 of 4 | Native OS TID: 2844

--- Joined thread team. Execution returned to serial master ---

--- Forking a team of 4 threads ---

[Master] Logical Rank: 0 of 4 | Native OS TID: 13764

[Worker] Logical Rank: 3 of 4 | Native OS TID: 16140

[Worker] Logical Rank: 1 of 4 | Native OS TID: 13764

[Worker] Logical Rank: 2 of 4 | Native OS TID: 20708

--- Joined thread team. Execution returned to serial master ---

--- Forking a team of 4 threads ---

[Master] Logical Rank: 0 of 4 | Native OS TID: 6468

[Worker] Logical Rank: 3 of 4 | Native OS TID: 32796

[Worker] Logical Rank: 1 of 4 | Native OS TID: 6468

[Worker] Logical Rank: 2 of 4 | Native OS TID: 34172

--- Joined thread team. Execution returned to serial master ---

--- Forking a team of 4 threads ---

[Master] Logical Rank: 0 of 4 | Native OS TID: 28116

[Worker] Logical Rank: 3 of 4 | Native OS TID: 17208

[Worker] Logical Rank: 1 of 4 | Native OS TID: 28116

[Worker] Logical Rank: 2 of 4 | Native OS TID: 1856

--- Joined thread team. Execution returned to serial master ---

Question 1.1: Scheduling

Thread IDs print in different orders because the operating system scheduler decides when each thread runs. The order depends on CPU availability, thread creation timing, scheduler priorities, interrupts, and context switches. On multicore CPUs, threads may execute simultaneously on different cores, so completion order is nondeterministic.

Question 1.2: Oversubscription

When P exceeds the number of physical CPU cores, multiple threads compete for the same cores. The OS performs more context switches, saving and restoring thread registers and stack state. This adds overhead and can cause cache thrashing because each thread replaces another thread's cached data. As a result, execution may become slower instead of faster.

Question 1.3: Barriers

An implicit barrier at the end of an OpenMP parallel region ensures that all worker threads finish before execution continues. Without the barrier, some threads could read incomplete data, overwrite shared results, or enter the next code block while other threads are still modifying shared variables. This can cause race conditions and inconsistent results.

Question 1.4: Architectural Mapping

Hardware execution thread / SMT thread: A logical CPU context provided by the processor. Multiple SMT threads share physical-core resources such as execution units, cache, and memory bandwidth.

Operating-system kernel thread: A schedulable software thread managed by the operating system. It can run on any available logical CPU.

Green or virtual thread: A lightweight thread managed by a runtime or language library. Many virtual threads may be multiplexed onto a smaller number of OS threads. They are efficient for I/O, but CPU-bound work still depends on available hardware and runtime limitations.